

Section 02: Labs

Lab 01: Inspecting Table Size

The screenshot shows a PostgreSQL query editor with a query to inspect table sizes. The query is as follows:

```

1  /*Lab 01*/
2  -- 1. Check table vs index size
3  SELECT
4      relname AS object_name,
5      pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
6      pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,
7      pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
8  FROM pg_class fed
9  JOIN pg_namespace n ON n.oid = fed.relnamespace
10 WHERE fed.relkind = 'r' AND n.nspname = 'public'
11 ORDER BY pg_total_relation_size(fed.oid) DESC;

```

The output of the query is shown in a table with 4 columns: object_name, data_size, index_size, and total_size. The results are as follows:

object_name	data_size	index_size	total_size
rental	1232 kB	1120 kB	2352 kB
payment	896 kB	920 kB	1816 kB
film	736 kB	200 kB	936 kB
film_actor	272 kB	216 kB	488 kB
inventory	232 kB	208 kB	440 kB
customer	96 kB	112 kB	208 kB
address	88 kB	64 kB	152 kB

The output also shows "Total rows: 16" and "Query complete 00:00:01.123".

Tables like **payment** and **customer** have an index footprint larger than their raw data footprint. This happens because they carry multiple indexes (Primary Keys, Foreign Keys, unique constraints) to optimize search queries.

While high index coverage speeds up read performance (**SELECT**), it introduces write overhead (**INSERT**, **UPDATE**, **DELETE**) as Postgres must update the index structures continuously.

Lab 02: The Impact of a New Index

The screenshot shows a PostgreSQL query editor with a query to check the size of the 'payment' table and its index. The query is as follows:

```

13  /*Lab 02*/
14  -- Step 1: Check current size of 'payment' indexes
15  SELECT pg_size_pretty(pg_indexes_size('payment'));
16  -- Step 2: Create a heavy index on payment_date
17  CREATE INDEX idx_payment_date_long
18  ON payment (payment_date, amount, customer_id);
19
20  -- Step 3: Check size again
21  SELECT pg_size_pretty(pg_indexes_size('payment'));

```

The output of the query is shown in a table with 1 column: pg_size_pretty. The result is as follows:

pg_size_pretty
1488 kB

The output also shows "Total rows: 1" and "Query complete 00:00:00.442".

The query result shows the total index size after creating `idx_payment_date_long` is **1488 kB**.

Compared to the initial size (**920 kB**), the newly added index consumed an additional **568 kB** of disk space.

Given PostgreSQL's standard block size of **8 KB** per Page, this new index consumed **568/8= 71 Pages (8KB)**.

Lab 03: Partial Index Practice

Create Partial Index

```
23 /*Lab 03*/
24 -- Create Partial Index
25 DROP INDEX idx_expensive_films
26
27 CREATE INDEX idx_expensive_films
28 ON film (replacement_cost)
29 WHERE replacement_cost > 25.00;
30 SELECT pg_size_pretty(pg_relation_size('idx_expensive_films'));
31
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

pg_size_pretty
16 kB

Successfully run. Total query runtime: 188 msec. 1 rows affected.

Total rows: 1 Query complete 00:00:00.188 CRLF Ln 30, Col 64

Create a hypothetical full index and compare.

```
32 -- Create the hypotheticalfull index
33 CREATE INDEX idx_full_replacement_cost
34 ON film (replacement_cost);
35
36 -- Check and Compare the real storage of 2 Indexes
37 SELECT
38     relname AS index_name,
39     pg_size_pretty(pg_relation_size(oid)) AS index_size,
40     pg_relation_size(oid) AS size_in_bytes
41 FROM pg_class
42 WHERE relname IN ('idx_expensive_films', 'idx_full_replacement_cost');
43
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

index_name	index_size	size_in_bytes
idx_expensive_films	16 kB	16384
idx_full_replacement_c...	40 kB	40960

Total rows: 2 Query complete 00:00:00.156 CRLF Ln 41, Col 14

Size Comparison: The Partial Index (**16 kB**) achieves a **60% reduction** in storage size compared to the Full Index (**40 kB**).

Page Consumption: The Partial Index consumes only **2 Pages (8KB)**, whereas the Full Index occupies **5 Pages (8KB)** on disk storage.

Reasoning: Out of 1,000 total rows in the `film` table, only a subset (around 326 rows) meets the `replacement_cost > 25.00` predicate. The Partial Index excludes non-matching rows entirely, resulting in a much smaller B-Tree structure.

Key Takeaways: This saves RAM/Buffer Cache footprint, accelerates query traversal, and eliminates maintenance overhead during `INSERT/UPDATE` operations on non-qualifying rows (`$\le 25.00$`).